

CLOUD SECURITY TOOLS AND TOOL OUTPUT REPORT

Professional Portfolio Edition - AWS CloudGoat / ScoutSuite / Pacu / IMDS Lab
Evidence

Prepared For	Enterprise Security Assessment Lab
Prepared By	Jagriti Banerjee
Module	04-cloud-security
Report Type	Cloud Security Tools and Tool Output Documentation
Environment	AWS CloudGoat lab, AWS CLI, Pacu, ScoutSuite, EC2 IMDS, S3, Billing Console
Safety Level	Authorized vulnerable cloud lab only - public portfolio version with sensitive values redacted

This document explains the cloud security tools used, the command output produced, the evidence captured, the security meaning of each output, and the remediation actions that would apply in a real cloud environment.

Table of Contents

Section	Topic
1	Executive Summary
2	Toolchain and Evidence Inventory
3	Output Handling and Redaction Standard
4	AWS CLI Identity and Authentication Output
5	IAM Permission Enumeration Output
6	Pacu Privilege Escalation Scan Output
7	S3 Public / Unauthenticated Access Test Output
8	ScoutSuite Audit Output
9	EC2 IMDS Credential Exposure Output
10	CloudGoat Cleanup and EC2 Verification Output
11	AWS Billing / Free Tier Verification Output
12	Remediation Roadmap
13	GitHub Publication Guidance
14	Appendix - Command Reference and Redaction Checklist

1. Executive Summary

This report documents the tools and tool outputs used in the Cloud Security module of the Enterprise Security Assessment Lab. The focus is on professional interpretation of command output, cloud security evidence, and safe GitHub publication practices.

The cloud work was performed in an authorized vulnerable lab environment and is designed to demonstrate practical cloud security skills: identity verification, IAM permission analysis, privilege escalation path review, public storage testing, automated cloud auditing, EC2 metadata credential exposure, resource cleanup, and billing verification.

Key outcomes

- AWS CLI authentication and identity verification were confirmed using a dedicated lab profile.
- IAM permissions were enumerated to understand the starting principal and policy exposure.
- Pacu was used to identify potential IAM privilege escalation methods in the vulnerable lab.
- S3 unauthenticated access behaviour was tested using `--no-sign-request`.
- ScoutSuite generated an AWS audit dashboard and specific findings were reviewed.
- EC2 IMDS temporary credential exposure was validated in the controlled lab path with secrets redacted.
- CloudGoat cleanup and billing verification were documented to show responsible cloud lab practice.

Important portfolio note

The screenshots in this report have been aligned and redacted for public publication. Access keys, secret keys, session tokens, AWS account identifiers, ARNs, public IPs, bucket names, and sensitive cloud resource identifiers are masked where necessary.

2. Toolchain and Evidence Inventory

Tool / Platform	Purpose in this lab	Evidence screenshot
AWS CLI	Authenticate, identify AWS principal, enumerate IAM/S3 resources	01-aws-state-authenticated-get-caller-identity.png
IAM CLI Commands	Review identity, attached policies, inline policies, policy simulator	05-iam-permissions-enumeration.png
Pacu	Cloud security exploitation framework for IAM privilege escalation	07-pacu-privilege-analysis-results.png
S3 CLI	Test authenticated and unauthenticated bucket access	11-s3-public-bucket-unauthenticated-access.png
ScoutSuite	Generate AWS audit dashboard and review specific misconfigurations	19-scoutsuite-findings-report-overview.png / 21-scoutsuite-findings-report-detail.png
curl + jq	Query EC2 instance metadata service and format temporary credentials	20-aws-ec2-metadata-credentials-stolen.png
CloudGoat	Deploy and destroy intentionally vulnerable AWS scenarios	13-cloudgoat-ec2-cleanup-verified.png
AWS Billing Console	Verify Free Tier and post-lab cleanup cost status	14-aws-free-tier-cleanup-check.png

These tools were selected because they represent common cloud security testing workflows: CLI-based identity review, automated misconfiguration analysis, storage exposure testing, metadata service validation, and operational cleanup verification.

3. Output Handling and Redaction Standard

Cloud screenshots often contain values that must not be published. This report uses sanitized command examples and redacted images. The goal is to show the technical workflow without exposing credentials or cloud account identifiers.

Sensitive value	Reason for redaction	Examples
Access keys and secret keys	Can be used to authenticate to AWS	AccessKeyId, SecretAccessKey
Session tokens	Temporary cloud credentials can still be valid	Token from IMDS role credentials
AWS account ID and ARN	Identifies the cloud account and principal path	Account, Arn, UserId
Bucket names	Can expose storage targets and naming patterns	s3://bucket-name
Public IPs and instance IDs	Can expose infrastructure targets	i-..., IPv4 addresses
Billing values / account profile	Private account and cost information	credits, account display name

Publication rule

For GitHub, show evidence and methodology, but never upload raw credentials, .aws files, Terraform state, Pacu session data, unredacted ScoutSuite HTML output, or screenshots containing valid secrets.

4. AWS CLI Identity and Authentication Output

Objective: Confirm that the AWS CLI was installed, configured, and authenticated using the dedicated CloudGoat/lab profile before deeper enumeration.

Area	Details
Tool / Component	AWS CLI / STS
Evidence type	Screenshot, command workflow, sanitized output interpretation
Publication status	GitHub-safe version with sensitive cloud values redacted

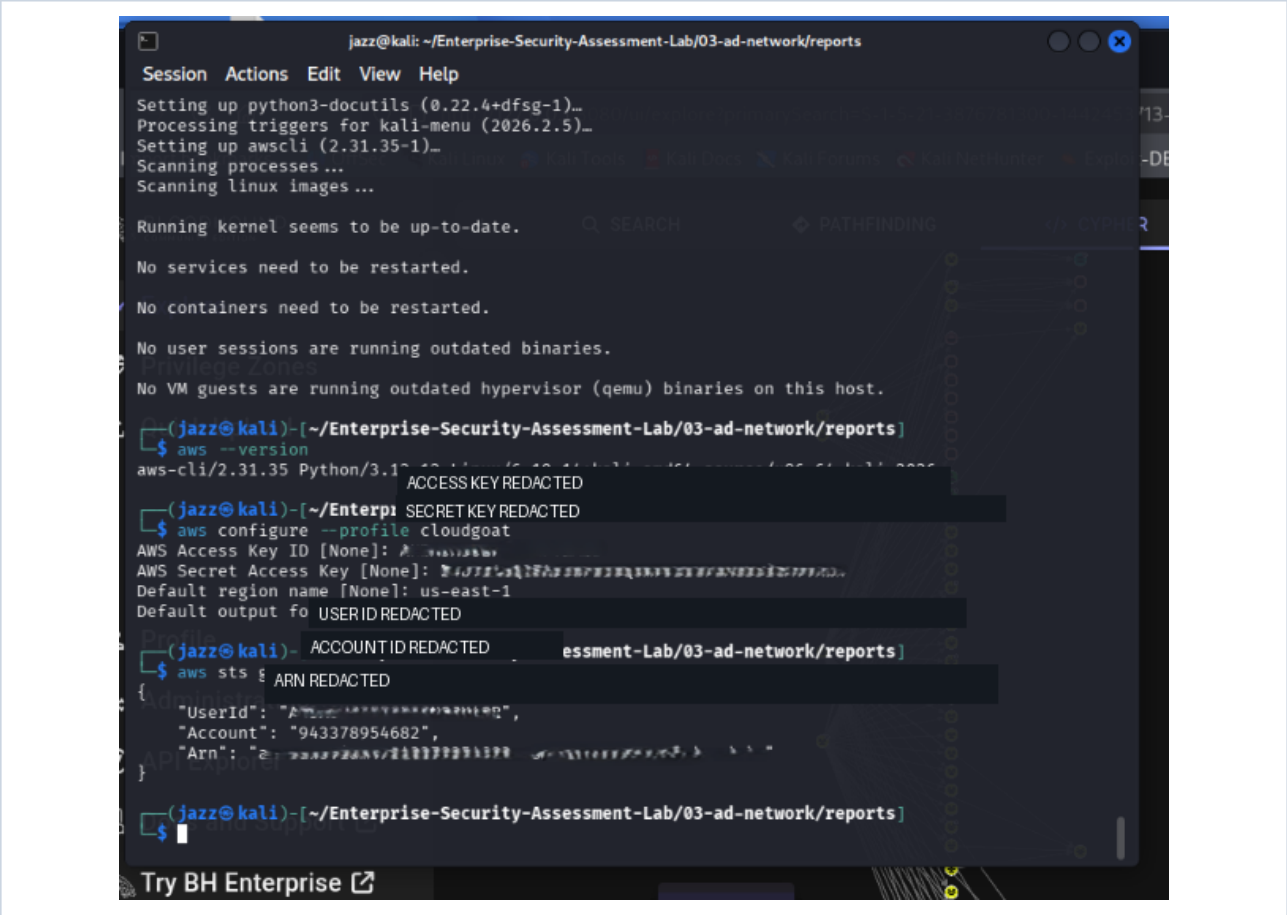


Figure 1 - Redacted AWS CLI authentication and STS caller identity output.

Command / workflow used

```
PROFILE=cloudgoat
REGION=us-east-1
aws --version
aws configure --profile $PROFILE
aws sts get-caller-identity --profile $PROFILE --region $REGION
```

Sanitized output style

```
{
  "UserId": "",
  "Account": "",
```

```
"Arn": "arn:aws:iam:::user/"  
}
```

Professional interpretation

The output proves that CLI authentication was working and that the active AWS principal could be identified. This is the baseline for every later cloud security action because IAM permissions and tool results are always tied to the current principal.

Risk if observed in production

If access keys are exposed publicly, an attacker may authenticate to the cloud account and perform actions allowed by the attached policies. Even low-privileged keys can become dangerous if privilege escalation paths exist.

Recommended remediation / hardening

- Avoid long-lived access keys where possible.
- Use MFA and short-lived credentials.
- Rotate any key that appears in screenshots or commits.
- Never commit .aws/credentials or CLI configuration files.
- Use IAM Access Analyzer and CloudTrail monitoring.

5. IAM Permission Enumeration Output

Objective: Identify what the starting lab identity can do by reviewing attached policies, inline policies, and effective policy documents.

Area	Details
Tool / Component	AWS CLI / IAM
Evidence type	Screenshot, command workflow, sanitized output interpretation
Publication status	GitHub-safe version with sensitive cloud values redacted

```

jazz@kali: ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
Session Actions Edit View Help
--profile $PROFILE
done | tee iam_inline_policy_documents.json
(jazz@kali)~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
$ for ARN in $(aws iam list-attached-user-policies --user-name "SAWS_USER" --profile $PROFILE
--query 'AttachedPolicies[].PolicyArn' --output text); do
echo "===== MANAGED POLICY: $ARN ====="
VERSION=$(aws iam get-policy --policy-arn "$ARN" --profile $PROFILE --query 'Policy.DefaultVer
sionId' --output text)
echo "Default version: $VERSION"
aws iam get-policy-version \
--policy-arn "$ARN" \
--version-id "$VERSION" \
--profile $PROFILE
done | tee iam_managed_policy_documents.json
===== MANAGED POLICY: arn:aws:iam::aws:policy/AdministratorAccess =====
Default version: v1
{
  "PolicyVersion": {
    "Document": {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Effect": "Allow",
          "Action": "*",
          "Resource": "*"
        }
      ]
    },
    "VersionId": "v1",
    "IsDefaultVersion": true,
    "CreateDate": "2015-02-06T18:39:46+00:00"
  }
}
(jazz@kali)~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
$

```

Figure 2 - IAM policy enumeration output showing managed policy review in the lab.

Command / workflow used

```

aws iam get-user --profile $PROFILE
aws iam list-attached-user-policies --user-name --profile $PROFILE
aws iam list-user-policies --user-name --profile $PROFILE
aws iam get-user-policy --user-name --policy-name --profile $PROFILE

```

Sanitized output style

```

Managed policy reviewed:
Policy statement example:
{
  "Effect": "Allow",

```

```
"Action": "*",  
"Resource": "*" }  
}
```

Professional interpretation

The output demonstrates IAM policy review and shows how an assessor can inspect whether an identity is overly permissive. In this lab, the evidence shows an administrator-style policy document, which is useful for understanding cloud privilege risk.

Risk if observed in production

Overly permissive IAM policies can allow direct compromise, resource creation, policy manipulation, credential creation, or privilege escalation across AWS services.

Recommended remediation / hardening

- Apply least privilege instead of wildcard actions.
- Separate read-only audit permissions from administrative permissions.
- Restrict iam:PassRole, iam:AttachUserPolicy, iam:CreateAccessKey and sts:AssumeRole.
- Use permission boundaries for lab users.
- Review attached and inline policies regularly.

6. Pacu Privilege Escalation Scan Output

Objective: Use Pacu to identify IAM privilege escalation possibilities for the current AWS principal in the intentionally vulnerable lab.

Area	Details
Tool / Component	Pacu
Evidence type	Screenshot, command workflow, sanitized output interpretation
Publication status	GitHub-safe version with sensitive cloud values redacted

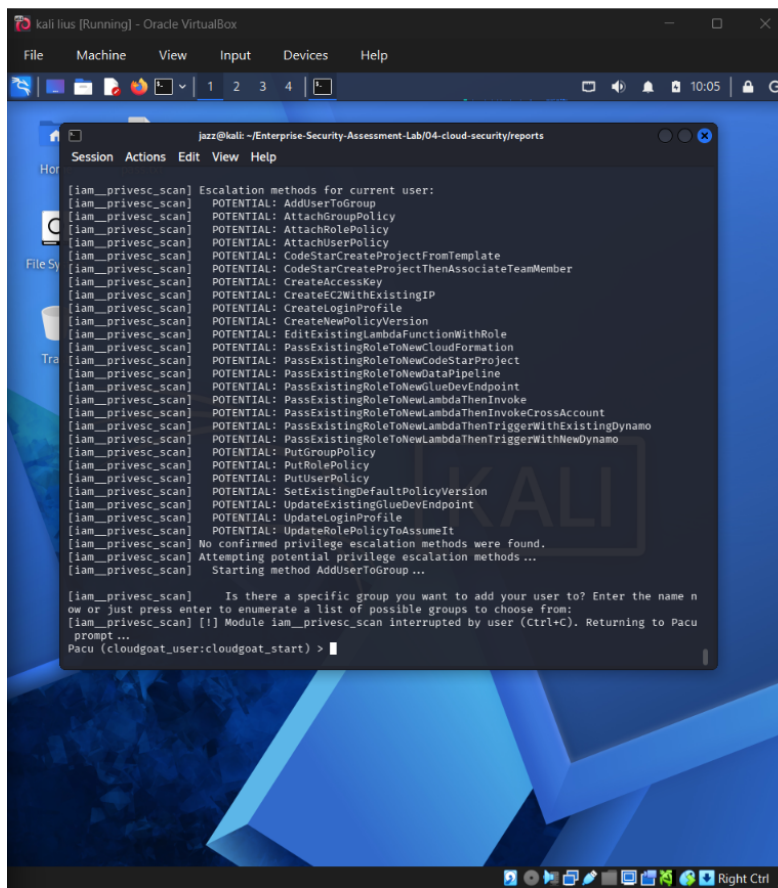


Figure 3 - Pacu IAM privilege escalation scan output listing potential methods.

Command / workflow used

```
pacu
import_keys cloudgoat
set_regions
us-east-1
whoami
run iam_privesc_scan
```

Sanitized output style

```
[iam_privesc_scan] Escalation methods for current user:
POTENTIAL: AddUserToGroup
```

```
POTENTIAL: AttachGroupPolicy
POTENTIAL: AttachUserPolicy
POTENTIAL: CreateAccessKey
POTENTIAL: PassExistingRoleToNewEC2ThenSSH
POTENTIAL: SetExistingDefaultPolicyVersion
```

Professional interpretation

Pacu found multiple privilege escalation methods that may be possible depending on the exact permissions of the current principal. This does not automatically mean every method succeeded; it means the permission model should be reviewed carefully.

Risk if observed in production

A low-privileged cloud user can sometimes become highly privileged by abusing policy management, role passing, access key creation, or compute-service role attachment.

Recommended remediation / hardening

- Continuously scan IAM permissions for escalation paths.
- Restrict policy attachment and policy version permissions.
- Limit iam:PassRole to approved service roles only.
- Use service control policies in multi-account environments.
- Alert on unusual IAM policy and access key changes.

7. S3 Public and Unauthenticated Access Test Output

Objective: Test whether S3 buckets created by the lab allow unauthenticated listing or object access.

Area	Details
Tool / Component	AWS CLI / S3
Evidence type	Screenshot, command workflow, sanitized output interpretation
Publication status	GitHub-safe version with sensitive cloud values redacted

```

jazz@kali: ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
$ aws s3 ls --profile cloudgoat
2026-06-07 10:18:33 BUCKET NAME REDACTED

(jazz@kali) ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
$ export BUCKET_NAME=cg-cardholder-data-bucket-...
$ echo $BUCKET_NAME
cg-cardholder-data-bucket-...

(jazz@kali) ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
$ aws s3 ls s3://$BUCKET_NAME --no-sign-request | tee s3_unsigned_list.txt
An error occurred (NoSuchBucket) when calling the ListObjectsV2 operation: The specified bucket does not exist

(jazz@kali) ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
$ aws s3 ls --profile cloudgoat
2026-06-07 10:18:34 cg-cardholder-data-bucket-...

(jazz@kali) ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
$ export BUCKET_NAME=$(aws s3 ls --profile cloudgoat | grep cardholder | awk '{print $3}')
$ echo $BUCKET_NAME
cg-cardholder-data-bucket-...

(jazz@kali) ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
$ echo "Testing unauthenticated S3 access for: $BUCKET_NAME"
Testing unauthenticated S3 access for: cg-cardholder-data-bucket-...

(jazz@kali) ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
$ aws s3api get-public-access-block --bucket $BUCKET_NAME --profile $PROFILE
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied

(jazz@kali) ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
$ aws s3api get-bucket-policy-status --bucket $BUCKET_NAME --profile $PROFILE

```

Figure 4 - S3 authenticated listing and no-sign-request public access test output with bucket names redacted.

Command / workflow used

```

aws s3 ls --profile $PROFILE
aws s3 ls s3:// --no-sign-request
aws s3api get-public-access-block --bucket --profile $PROFILE
aws s3api get-bucket-policy-status --bucket --profile $PROFILE

```

Sanitized output style

Testing unauthenticated S3 access for:
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied

Professional interpretation

The output shows a structured test for public S3 access. In the visible evidence, the no-sign-request test returns AccessDenied, so public bucket listing was tested but not confirmed from the shown output.

Risk if observed in production

Public S3 buckets can expose backups, source code, application logs, credentials, customer data, or configuration files.

Recommended remediation / hardening

- Enable S3 Block Public Access at account and bucket level.
- Avoid Principal "*" in bucket policies unless deliberately required.
- Review ACLs and bucket policies.
- Use AWS Config and Security Hub checks for public buckets.
- Log S3 access and investigate anonymous access attempts.

8. ScoutSuite AWS Audit Output

Objective: Run an automated AWS configuration audit and review the dashboard for service-level findings and control gaps.

Area	Details
Tool / Component	ScoutSuite
Evidence type	Screenshot, command workflow, sanitized output interpretation
Publication status	GitHub-safe version with sensitive cloud values redacted

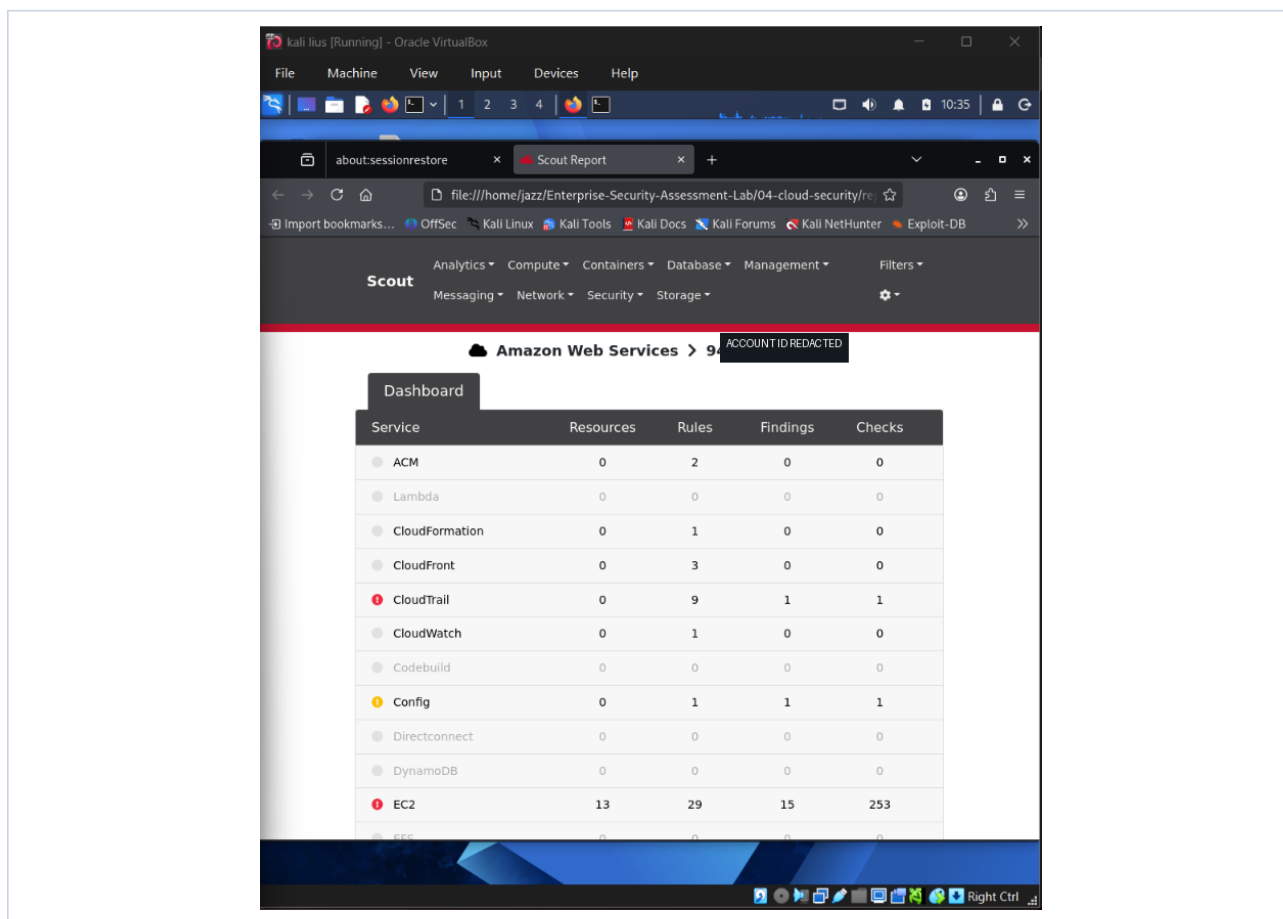


Figure 5 - ScoutSuite AWS audit dashboard with AWS account identifier redacted.

Command / workflow used

```
scout aws --profile $PROFILE --regions $REGION --no-browser --report-dir ./scoutsuite-report
find ./scoutsuite-report -name "*.html" | head -1
```

Sanitized output style

ScoutSuite generated an HTML report.
 Services reviewed: IAM, EC2, CloudTrail, Config, S3, CloudWatch and others.
 Dashboard showed findings and checks by service.

Professional interpretation

ScoutSuite provides a broad view of cloud configuration issues. The dashboard helps prioritize services with findings and supports deeper review of specific risks.

Risk if observed in production

Cloud misconfigurations often accumulate across services. Without automated auditing, exposed storage, weak logging, permissive security groups, and IAM risks can remain unnoticed.

Recommended remediation / hardening

- Run cloud audits regularly.
- Prioritize high-risk IAM, S3 and network findings.
- Enable AWS Config, CloudTrail, GuardDuty and Security Hub where appropriate.
- Track findings to closure.
- Avoid publishing raw ScoutSuite output if it contains account details.

9. ScoutSuite Specific Finding Output

Objective: Open a specific ScoutSuite finding to understand affected rules, checked resources, and severity context.

Area	Details
Tool / Component	ScoutSuite / Browser
Evidence type	Screenshot, command workflow, sanitized output interpretation
Publication status	GitHub-safe version with sensitive cloud values redacted

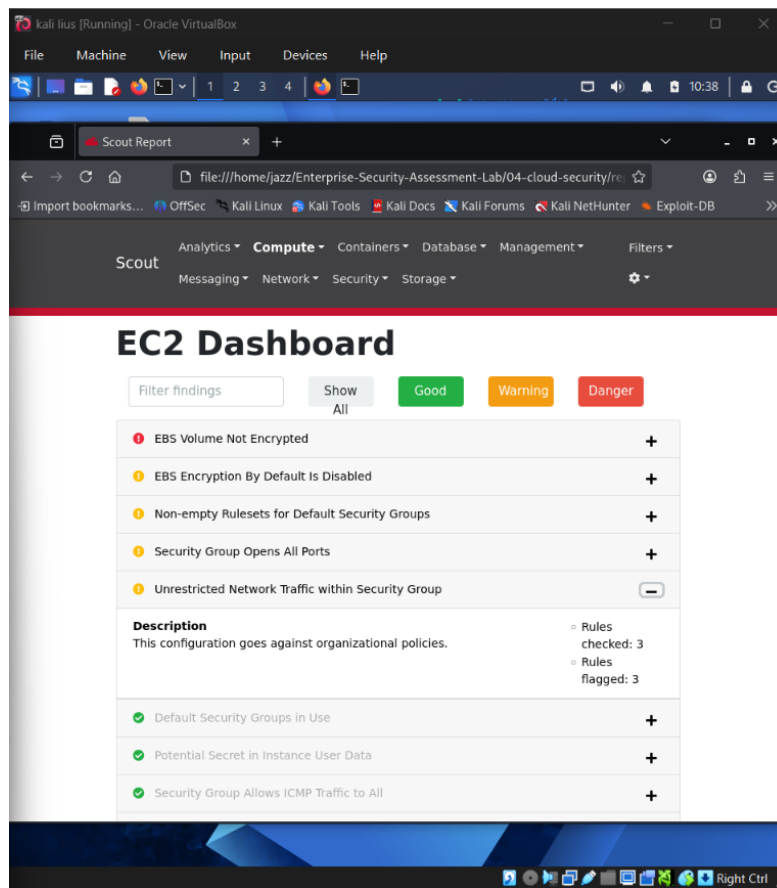


Figure 6 - ScoutSuite EC2 finding detail showing an unrestricted network traffic finding.

Command / workflow used

```
# Open generated ScoutSuite HTML report
# Navigate to Compute -> EC2 -> Findings
# Review expanded finding details and affected rules
```

Sanitized output style

```
Finding category: EC2 / Security Group configuration
Finding example: Unrestricted Network Traffic within Security Group
Rules checked:
Rules flagged:
```

Professional interpretation

The detailed ScoutSuite page shows how a single finding can be expanded and documented. This is useful for report evidence because it connects a dashboard count to an actionable control issue.

Risk if observed in production

Overly permissive security groups can expose services to broad networks and increase the attack surface of EC2 workloads.

Recommended remediation / hardening

- Restrict inbound rules to required ports and trusted sources.
- Avoid 0.0.0.0/0 exposure unless explicitly needed.
- Use security group reviews as part of change management.
- Monitor new or modified security group rules.
- Apply network segmentation.

10. EC2 IMDS Credential Exposure Output

Objective: Validate whether temporary EC2 instance profile credentials could be retrieved through the vulnerable metadata exposure path in the lab.

Area	Details
Tool / Component	curl / jq / EC2 Instance Metadata Service
Evidence type	Screenshot, command workflow, sanitized output interpretation
Publication status	GitHub-safe version with sensitive cloud values redacted

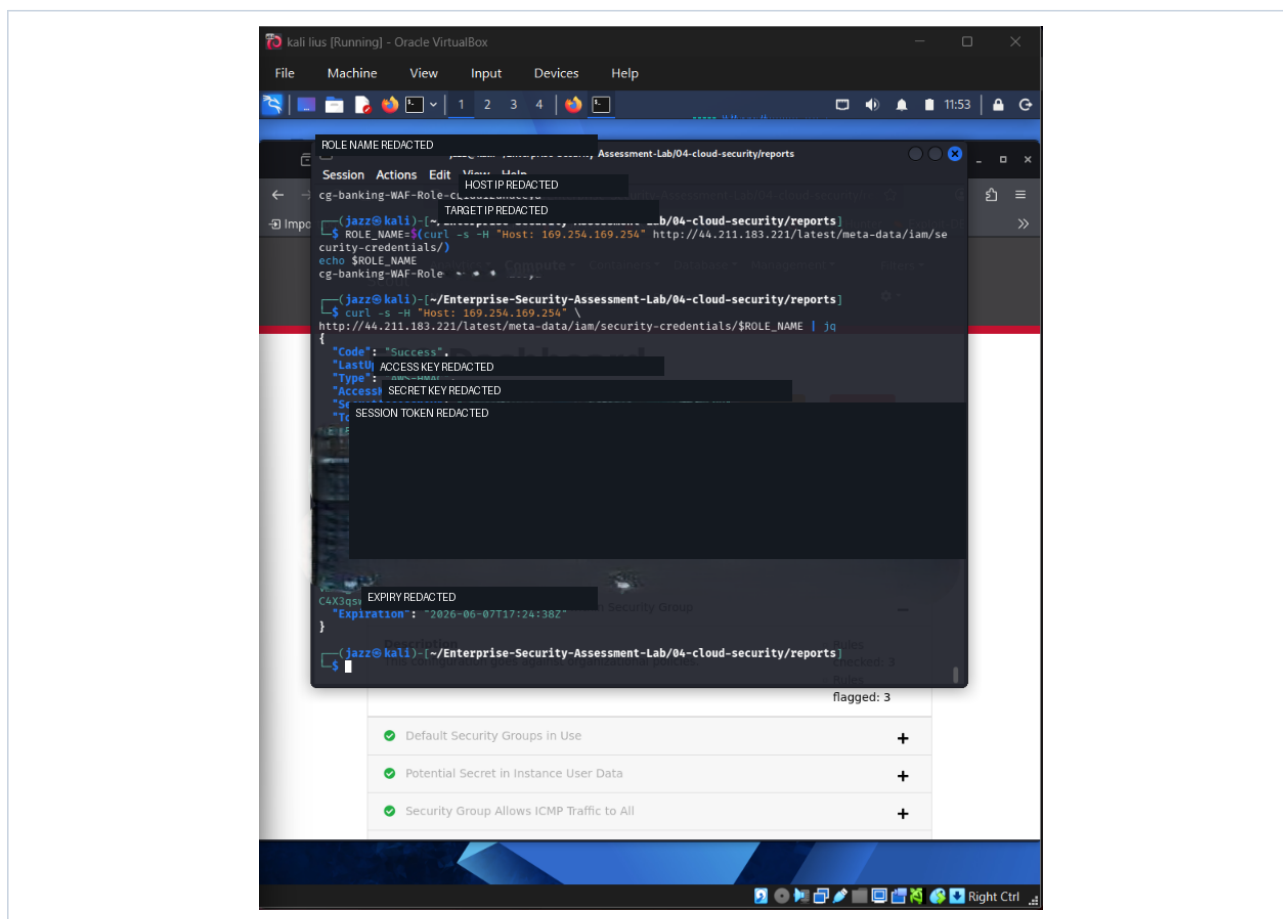


Figure 7 - EC2 metadata service credential retrieval output with secrets fully redacted.

Command / workflow used

```
# Metadata service endpoint
curl -s http://169.254.169.254/latest/meta-data/

# Discover attached role
curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/

# Retrieve temporary credentials
curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/ | jq
```

Sanitized output style

```
{
  "Code": "Success",
  "Type": "AWS-HMAC",
  "AccessKeyId": "",
  "SecretAccessKey": "",
  "Token": "",
  "Expiration": ""
}
```

Professional interpretation

The output confirms that temporary role credentials were retrievable from IMDS in the controlled lab path. This is a high-value finding because temporary credentials can be used until expiry if they are not contained.

Risk if observed in production

If an attacker reaches IMDS through SSRF, command execution, or a compromised instance, they may steal temporary role credentials and access AWS resources allowed by the instance profile.

Recommended remediation / hardening

- Require IMDSv2.
- Harden applications against SSRF.
- Avoid attaching powerful roles to EC2 instances.
- Apply least privilege to instance profiles.
- Monitor use of temporary credentials from unusual locations.

11. CloudGoat Cleanup and EC2 Verification Output

Objective: Verify that vulnerable lab resources were destroyed after testing and that EC2 instances were terminated.

Area	Details
Tool / Component	CloudGoat / AWS CLI EC2
Evidence type	Screenshot, command workflow, sanitized output interpretation
Publication status	GitHub-safe version with sensitive cloud values redacted

```

jazz@kali: ~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports
Session Actions Edit View Help SCENARIO ID REDACTED
Destroy "iam_privesc_by_attachment_cgidt207ykezfj"? [y/n]: y

No terraform.tfstate file was found in the scenario instance's terraform directory, so "terraform destroy" will not be run.

Successfully destroyed iam_privesc_by_attachment_cgidt207ykezfj.
Scenario instance files have been moved to /home/jazz/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/trash/iam_privesc_by_attachment_cgidt207ykezfj

(jazz@kali)-[~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports]
$ ls -lh cloudgoat-attack-key.pem
-r----- 1 jazz jazz 1.7K Jun  6 20:54 cloudgoat-attack-key.pem

(jazz@kali)-[~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports]
$ rm -f cloudgoat-attack-key.pem

(jazz@kali)-[~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports]
$ aws ec2 describe-instances \
  --profile cloudgoat \
  --region us-east-1 \
  --query "Reservations[*].Instances[*].[InstanceId,State.Name,Tags[?Key=='Name'].Value[0]]" \
  --output table

DescribeInstances
+-----+-----+-----+-----+
| i-0151b3958f5679939 | terminated | CloudGoat | INSTANCE DETAIL REDACTED | local-security-server E
C2 Instance |
| i-04944444444444444 | terminated | None | INSTANCE ID REDACTED | IP REDACTED
+-----+-----+-----+-----+

(jazz@kali)-[~/Enterprise-Security-Assessment-Lab/04-cloud-security/reports]
$

```

Figure 8 - CloudGoat scenario destroyed and EC2 instance cleanup verified with instance details redacted.

Command / workflow used

```

cloudgoat destroy
rm -f cloudgoat-attack-key.pem
aws ec2 describe-instances --profile cloudgoat --region us-east-1 --query "Reservations[*].Instances[*].[InstanceId,State.Name,Tags[?Key=='Name'].Value[0]]" --output table

```

Sanitized output style

```

Successfully destroyed .
EC2 cleanup verification:
  terminated

```

Professional interpretation

The output proves responsible cloud lab handling. Cleanup verification is a critical operational control because vulnerable labs can create cost and security exposure if left running.

Risk if observed in production

Unremoved lab resources can generate charges, expose services, leave credentials active, and expand attack surface.

Recommended remediation / hardening

- Destroy scenarios immediately after testing.
- Verify EC2, S3, IAM and CloudFormation cleanup.
- Delete temporary keys.
- Use short-lived lab accounts or sandbox accounts.
- Run billing checks after every cloud lab session.

12. AWS Free Tier and Billing Verification Output

Objective: Confirm the billing dashboard and Free Tier usage were checked after cloud lab activity.

Area	Details
Tool / Component	AWS Billing Console
Evidence type	Screenshot, command workflow, sanitized output interpretation
Publication status	GitHub-safe version with sensitive cloud values redacted

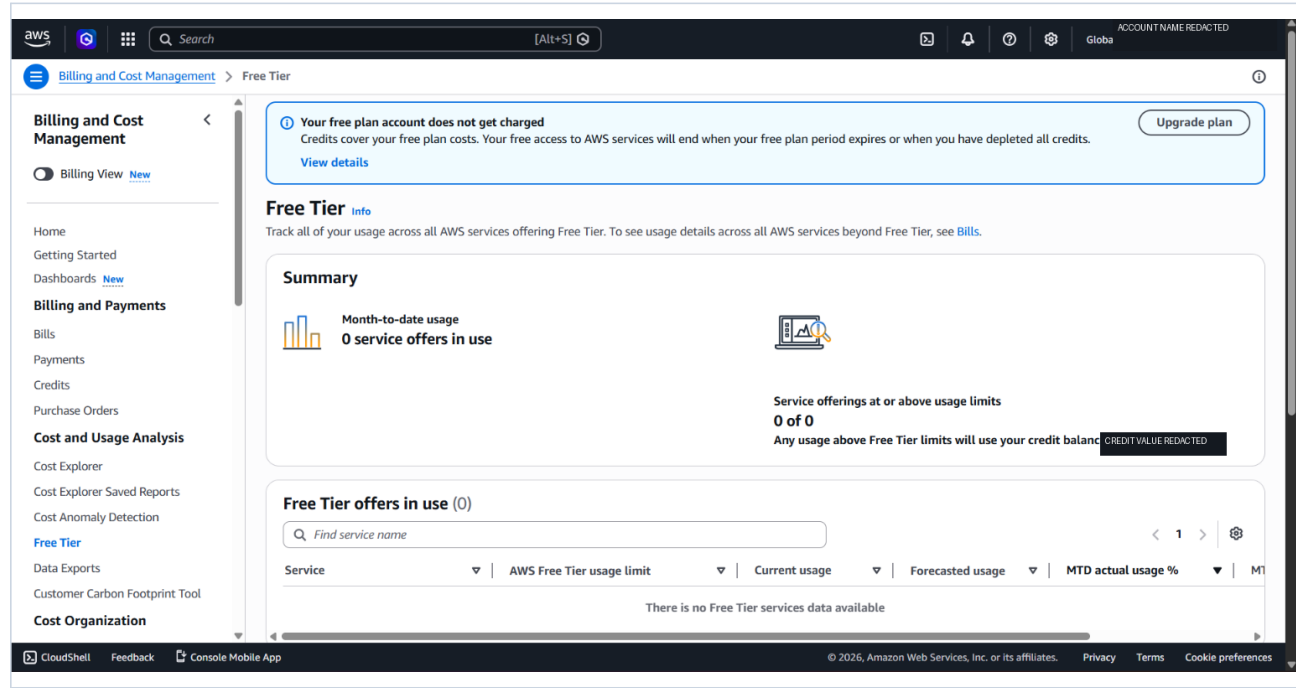


Figure 9 - AWS Billing and Free Tier dashboard checked after cleanup with account details redacted.

Command / workflow used

AWS Console -> Billing and Cost Management -> Free Tier / Bills
Review active services, month-to-date usage, and unexpected charges.

Sanitized output style

Free Tier summary reviewed.
Service offers in use: 0
Unexpected active resource usage: not observed in screenshot evidence.

Professional interpretation

Billing verification demonstrates mature cloud lab discipline. It shows the assessor understands that cloud security includes cost control and lifecycle management, not only exploitation.

Risk if observed in production

Cloud labs can create unexpected cost if EC2 instances, Elastic IPs, NAT Gateways, storage buckets, logs, or IAM roles remain active.

Recommended remediation / hardening

- Create AWS Budgets and alerts.
- Use a dedicated cloud lab account.
- Destroy all test resources after use.
- Check billing after each lab session.
- Tag resources and apply automated cleanup where possible.

13. Remediation Roadmap

Priority	Control Area	Recommended Action
P0	Credential exposure	Rotate any exposed access keys, session tokens, or role credentials immediately.
P0	IMDS exposure	Require IMDSv2 and reduce instance profile permissions.
P1	IAM privilege escalation	Restrict iam:PassRole, policy attachment, access key creation, and role assumption.
P1	S3 exposure	Enable Block Public Access and review bucket policies/ACLs.
P1	Network exposure	Review ScoutSuite network findings and restrict broad ingress rules.
P2	Audit coverage	Enable AWS Config, CloudTrail, GuardDuty and Security Hub where suitable.
P2	Cost control	Use budgets, alerts, tags, and post-lab cleanup verification.

14. GitHub Publication Guidance

Recommended repository location:

```
04-cloud-security/reports/Cloud_Security_Tools_and_Tool_Output_Report_Professional.pdf
```

README link:

```
[Cloud Security Tools and Tool Output Report](./reports/Cloud_Security_Tools_and_Tool_Output_Report_Professional.pdf)
```

Do not upload

- .aws/credentials or .aws/config files
- Terraform state files or tfvars containing secrets
- Raw Pacu session databases
- Unredacted ScoutSuite reports
- Private keys, PEM files, or generated SSH keys
- Screenshots containing AccessKeyId, SecretAccessKey, Token, Account, Arn, bucket names, or public IPs

15. Appendix - Command Reference

```
# Identity
aws sts get-caller-identity --profile cloudgoat --region us-east-1

# IAM user and policies
aws iam get-user --profile cloudgoat
aws iam list-attached-user-policies --user-name --profile cloudgoat
aws iam list-user-policies --user-name --profile cloudgoat

# S3 public access testing
aws s3 ls s3:// --no-sign-request
aws s3api get-public-access-block --bucket --profile cloudgoat

# Pacu
pacu
import_keys cloudgoat
run iam__privesc_scan

# ScoutSuite
scout aws --profile cloudgoat --regions us-east-1 --no-browser --report-dir ./scoutsuite-report

# EC2 IMDS
curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/
curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/ | jq

# Cleanup
cloudgoat destroy
aws ec2 describe-instances --profile cloudgoat --region us-east-1
```

Final legal and ethical notice

This report documents only authorized testing performed in a personal vulnerable cloud lab. The techniques described must not be used against third-party cloud accounts or production environments without explicit written permission.